

后端第4题

Task 1

Q 1

•

	数组	集合
长度	固定，创建后不能改	可变，自动扩容
能存什么	基本类型 + 引用类型	只能存引用类型（对象）
有没有现成方法	几乎没有（只能靠下标访问）	大量方法（add、remove、contains...）
举例	<code>int[] arr</code>	<code>ArrayList</code> 、 <code>HashSet</code> 、 <code>HashMap</code>

• 对图片的分析：

1. 第一层：最顶端的接口

•

名称	颜色	是什么
<code>Iterable</code>	紫（接口）	所有集合的"老祖宗"，提供遍历能力
<code>Collection</code>	紫（接口）	所有"单列集合"（一个元素一个元素存）的父接口
<code>Map</code>	紫（接口）	所有"键值对集合"的父接口（独立体系）

2. 第二层：Collection 的两个子接口

•

名称	颜色	特点
<code>List</code>	紫（接口）	有序、可重复
<code>Set</code>	紫（接口）	无序、不可重复

3.

接口	实现类（黄框）	底层结构	特点
List	<code>ArrayList</code>	数组	查询快，增删慢
List	<code>LinkedList</code>	链表	增删快，查询慢

接口	实现类 (黄框)	底层结构	特点
List	Vector	数组	线程安全 (老版本)
Set	HashSet	哈希表	无序, 去重, 查询极快
Set	TreeSet	红黑树	自动排序
Set	LinkedHashSet	哈希表+链表	有序 (按插入顺序) 去重
Map	HashMap	哈希表	键值对, 最常用
Map	TreeMap	红黑树	按键排序
Map	LinkedHashMap	哈希表+链表	按插入顺序

你的需求	选哪个	原因
要一串有顺序、可重复的数据, 常按下标查	ArrayList	查询快
要频繁在中间增删	LinkedList	增删快
要自动去重, 不关心顺序	HashSet	哈希去重
要去重还要排序	TreeSet	自动排序
要"键→值"对应关系	HashMap	键值对
要键值对还要按键排序	TreeMap	按键排序

Q 2

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
        list.add(4);

        // 增强 for 循环遍历
        for (Integer num : list) {
```

```
        System.out.println(num);
    }
}
}
```

Q 3

1. 什么是匿名内部类？

没有名字的内部类，当场定义、当场创建对象、用完就丢。通常用于"只需要一次性的、实现了某接口的类"的场景，省去了单独写一个类的麻烦。

2. 什么是函数式接口？

只有一个抽象方法的接口。比如 Consumer（只有 accept 一个抽象方法）。它的意义在于：可以用 lambda 表达式简洁地实现它。

3. Lambda 表达式用法总结

- 语法：(参数) -> { 方法体 }
- 参数类型可以省略（自动推断）
- 只有一个参数时，() 可以省略
- 方法体只有一句话时，{} 和 return 可以省略
- 只能用在函数式接口上（这是关键前提）
-

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
        list.add(4);

        // 用 lambda 表达式遍历（最简洁写法）
        list.forEach(integer -> System.out.println(integer));
    }
}
```

```
}  
}
```

Task 2

Q 4

```
import java.util.HashMap;  
import java.util.Map;  
  
public 类 (Main)  
interface Repository<T> {  
    void save(T data);  
    T getById(int id);  
}  
  
// 实现类 (真仓库)  
class MyRepository<T> implements Repository<T> {  
  
    private Map<Integer, T> storage = new HashMap<>(); // 编号柜子  
    private int nextId = 0; // 发号机  
  
    @Override  
    public void save(T data) {  
        storage.put(nextId, data);  
        nextId++;  
    }  
  
    @Override  
    public T getById(int id) {  
        return storage.get(id);  
    }  
}  
  
// 用户类  
class User {  
    private String name;  
    private int age;
```

```
public User(String name, int age) {
    this.name = name;
    this.age = age;
}

@Override
public String toString() {
    return "User{name=' " + name + "', age=" + age + "'}";
}
}
```

// 主类 (唯一 public 类, 文件名必须是 Main.java)

```
public class Main {
    public static void main(String[] args) {

        // 仓库1: 存文字
        MyRepository<String> strRepo = new MyRepository<>();
        strRepo.save("苹果");
        strRepo.save("香蕉");
        strRepo.save("橙子");
        System.out.println("=== 文字仓库 ===");
        for (int id = 0; id < 3; id++) {
            System.out.println("id=" + id + ", 内容=" + strRepo.getById(id));
        }

        // 仓库2: 存用户
        MyRepository<User> userRepo = new MyRepository<>();
        userRepo.save(new User("张三", 20));
        userRepo.save(new User("李四", 25));
        System.out.println("=== 用户仓库 ===");
        for (int id = 0; id < 2; id++) {
            System.out.println("id=" + id + ", 内容=" + userRepo.getById(id));
        }

        // 仓库3: 存数字
        MyRepository<Integer> intRepo = new MyRepository<>();
        intRepo.save(100);
        intRepo.save(200);
        intRepo.save(300);
        System.out.println("=== 数字仓库 ===");
        for (int id = 0; id < 3; id++) {
```

```
        System.out.println("id=" + id + ", 内容=" + intRepo.getById(id));
    }
}
}
```

Q5

```
public enum CharacterType {
    SABER, ARCHER, CASTER
}

public interface Character {
    void attack();
}
```

```
public class Saber implements Character {
    private String name;
    private String skill;

    public Saber() {
        this.name = "阿尔托莉雅";
        this.skill = "Excalibur";
    }

    @Override
    public void attack() {
        System.out.println("[剑士] " + name + " 使用 " + skill + " 发动攻击!");
    }
}
```

```
public class Archer implements Character {
    private String name;
    private String skill;

    public Archer() {
        this.name = "卫宫";
        this.skill = "Unlimited Blade Works";
    }
}
```

```

@Override
public void attack() {
    System.out.println("[弓兵] " + name + " 使用 " + skill + " 发动攻击!");
}
}

```

```

public class Caster implements Character {
    private String name;
    private String skill;

    public Caster() {
        this.name = "美狄亚";
        this.skill = "Rho Aias";
    }

    @Override
    public void attack() {
        System.out.println("[法师] " + name + " 使用 " + skill + " 发动攻击!");
    }
}

```

```

public class CharacterFactory {
    public static Character create(CharacterType type) {
        switch (type) {
            case SABER:
                return new Saber();
            case ARCHER:
                return new Archer();
            case CASTER:
                return new Caster();
            default:
                throw new IllegalArgumentException("未知的角色类型: " + type);
        }
    }
}

```

```

public class Main {
    public static void main(String[] args) {

```

```
Character saber = CharacterFactory.create(CharacterType.SABER);  
Character archer = CharacterFactory.create(CharacterType.ARCHER);  
Character caster = CharacterFactory.create(CharacterType.CASTER);  
  
saber.attack();  
archer.attack();  
caster.attack();  
}  
}
```

运行结果在库里